

renders a neutral change harmless. The interaction of this re-evaluation with diverse control flows is developed in Section 5.1.3.

Algorithm 3 Reactive notification

```

1  function notify(ctx, keys)
2      affected  $\leftarrow \emptyset$ 
3      for fiber in all_fibers do
4          for key in keys do
5              if key  $\in$  fiber.inject and fiber.ctx[@@isolate][key] = ctx[@@isolate][key] then
6                  refresh(fiber)
7                  affected  $\leftarrow$  affected  $\cup$  {fiber}
8                  break
9      return affected

```

A binding counts as available to a dependent only while the fiber that installed it is **ACTIVE**, so refresh resolves each declared key against an active provider rather than against the store alone. This is the *provided by* relation of Definition 53, and it is what makes a withdrawal visible to dependents one step before it happens: a provider that has entered **UNLOADING** has stopped providing, so its dependents recompute an unsatisfied target view and begin their own teardown while its bindings are all still in place.

Isolation and interception. The two operations do structurally the same thing: each derives a child context that adjusts one inherited table for key, leaving the parent untouched, so recovery is implicit: discarding the child context suffices, with no explicit inverse to run. `ctx.isolate(key, realm)` overrides the realm mapping ρ with `realm`, or a freshly generated symbol by default (realizing `isolate`, Definition 25), so two contexts that assign different symbols to the same key resolve to independent bindings. `ctx.intercept(key, metadata)` merges `metadata` into the interception table ι (realizing `intercept`, Definition 27): following that definition, the new metadata is combined with whatever the context already carries for key and takes priority over it.

5.1.3. Component Lifecycle

A component is instantiated as a *fiber* by `ctx.use`. This section gives the fiber (introduced in Section 5.1) operational meaning as the inertial state machine of Section 4.4. Two fields drive the algorithm below: `fiber.parent`, the parent context of `fiber.ctx` that forms the component hierarchy (the recursive structure of Γ_∞ , Section 3.3.1), and `fiber.inertia`, a handle to the in-flight asynchronous transition (or null if idle).

Algorithm 4 shows component instantiation. A *component* pairs a coeffect specification `component.inject` (d) with an effect function `component.apply`; instantiation binds the component's config into `fiber.apply` (Line 9), the config-applied effect function (e) that the lifecycle then runs. The `callback` function (Line 2) is the effect tracked in the *parent* fiber: when executed, it initiates the child's lifecycle by calling `refresh` (Algorithm 5); when reverted, it forces the child's target to $\perp$ and triggers `unload`. This is the instantiation primitive of Definition 52, with `callback` as its O-Insert and the closure `callback` returns as its O-Retire: an instantiation is an ordinary tracked effect of the parent, so unloading a parent cascades to its children.

Algorithm 4 Component instantiation

```
1 function use(ctx, component, config)
2   function callback()
3     refresh(fiber)
4     return function()
5       fiber.target  $\leftarrow \perp$ 
6       unload(fiber)
7   fiber  $\leftarrow$  Fiber(parent: ctx, inject: component.inject)
8   fiber.ctx  $\leftarrow$  ctx[fiber  $\mapsto$  fiber]
9   fiber.apply  $\leftarrow$  ()  $\mapsto$  component.apply(fiber.ctx, config)
10  ctx.effect(callback)
11  return fiber
```

Algorithm 5 realizes the inertial state machine of Section 4.4, in which `reload` and `unload` are inertial: once entered, a transition runs to completion before the system responds to a target-state change. It uses two auxiliary lookups over the coeffect store: `resolve(inject)` returns the bindings the declared keys currently resolve to, and `provided(fiber)` returns the keys whose binding this fiber installed. The `refresh` function recomputes `fiber.target` from the coeffect store and, if the fiber is not already in a transition, initiates either a `reload` or `unload` task². The `reload` function records the current target and executes the component’s effect function `apply`. Upon completion, it checks whether the target still matches: if so, the fiber enters `ACTIVE`; if not (regardless of whether the new target is $\perp$ or a different set of providers), it chains into `unload`. Symmetrically, `unload` reverts all tracked effects in LIFO order and then either enters `INACTIVE` or chains into `reload`. This mutual recursion implements the inertial property: once a transition begins, it completes before any new transition can start.

Algorithm 5 Component lifecycle

```
1 function refresh(fiber)
2   target  $\leftarrow$  target( $\gamma$ ,  $n$ )
3   if target = fiber.target then return
4   fiber.target  $\leftarrow$  target
5   if fiber.inertia then return
6   if target  $\neq \perp$  then
7     fiber.state  $\leftarrow$  LOADING
8     fiber.inertia  $\leftarrow$  create_task(reload(fiber))
9   else
10    fiber.state  $\leftarrow$  UNLOADING  $\triangleright$  out of service before any inverse is scheduled
11    fiber.inertia  $\leftarrow$  create_task(unload(fiber))
12 async function reload(fiber)
13   target0  $\leftarrow$  fiber.target
14   fiber.committed  $\leftarrow$  resolve(fiber.inject)  $\triangleright$  commit the view
15   recover  $\leftarrow$  await execute(fiber.apply, ()  $\mapsto$  fiber.target = target0)
16   fiber.dispose  $\leftarrow$  recover  $\circ$  fiber.dispose
```

²`create_task` schedules an async function to run concurrently and returns a handle to it (stored in `fiber.inertia`). We write it explicitly for language independence: with eager scheduling (e.g., TypeScript promises), the call is implicit and the returned promise is the handle, whereas with lazy scheduling (e.g., Python coroutines, Rust futures) the host must spawn the task for it to progress.

```

17   if fiber.target = target0 then
18       | fiber.state ← ACTIVE
19       | notify(fiber.ctx, provided(fiber))
20       | fiber.inertia ← null
21   else
22       | fiber.state ← UNLOADING
23       | fiber.inertia ← create_task(unload(fiber))
24   async function unload(fiber)
25       await all(notify(fiber.ctx, provided(fiber)).map(f ↦ f.await())) ▷ drain dependents
26       await fiber.dispose()
27       fiber.dispose ← id
28       fiber.committed ← ⊥
29       if fiber.target = ⊥ then
30           | fiber.state ← INACTIVE
31           | fiber.inertia ← null
32       else
33           | fiber.state ← LOADING
34           | fiber.inertia ← create_task(reload(fiber))

```

`fiber.target` is computed by resolving each declared key against the current coeffect store and tupling the uid of the fiber that provides it, so it is a digest of $\text{target}(\gamma, n)$ (Definition 53). Identifying a binding by its provider rather than by its value is what makes a single comparison against the recorded target sufficient: a uid is drawn fresh and never reused, so a provider that is replaced cannot be mistaken for the one it replaced, even when the two provide equal values. Since `notify` (Section 5.1.2) recomputes the target on every coeffect change, a fiber reloads precisely when one of its declared keys comes to be provided by a different fiber. A provider that overwrites its own binding in place is therefore not observed; a component that wants its replacement to propagate withdraws the binding and installs it afresh.

The algorithm operates at two complementary levels. At the *transition* level, `reload` and `unload` check the target at completion, enabling inertial chaining across transitions. At the *iteration* level within each transition, the effect execution (Algorithm 1) checks the target at each iteration boundary, enabling partial rollback within a single transition. These two mechanisms correspond to the inter-transition chaining of Section 4.4 and the intra-transition staleness check that Theorem 71 rests on.

Three lines carry the coeffect ordering of Theorem 70, and where each of them sits is what makes the ordering hold. `reload` commits the resolved view at Line 14 and `unload` discards it only after every inverse has run, so a fiber reads the same bindings for as long as it is loaded, its own teardown included. `refresh` marks the fiber UNLOADING at Line 10 before the transition task is created, which is the L-Leave step: the fiber stops providing, and the dependents recompute against that before any of its inverses is scheduled. `unload` then waits at Line 25 for each notified dependent to reach INACTIVE, which is the guard on L-Unload; `notify` admits a dependent only when its declared key resolves to the same realm symbol as the provider's, which is the runtime form of the guard's demand that the dependent see the key *from this fiber* rather than merely declare it. The wait sits ahead of the whole recovery rather than inside one of the inverses being waited on, since `fiber.dispose` initiates a fiber's effects concurrently and a wait placed within one of them would leave the rest unordered. Termination follows Theorem 73: a fiber only ever waits on dependents that have already stopped being satisfiable, and a dependent that is itself

a provider waits the same way for its own, so the provider graph is traversed on demand rather than analyzed in advance.

5.1.4. Context Access

The coeffect operations of Section 5.1.2 form a *reflective* API: a coeffect is written with `ctx.set(key, value)` and read with `ctx.get(key)`, both keyed by name. Cordis layers a second, more native way to extend and consume the context on top of this reflective API: *property access*. A component can access a coeffect as the property `ctx[key]`, as if it were native structure of the context, rather than through a method call. In TypeScript, Cordis realizes this with a Proxy whose get trap mediates every property access. Algorithm 6 shows how a context resolves such an access to a coeffect, atop the primitive get of Section 5.1.2.

Algorithm 6 Proxy-mediated context access

```

1  function resolve(ctx, key)
2      fiber ← ctx.fiber
3      repeat
4          if key ∈ fiber.committed then return fiber.committed[key]
5          if key ∈ fiber.inject then throw INACTIVE_ACCESS
6          if fiber = root then throw UNDECLARED_ACCESS
7          fiber ← fiber.parent.fiber

```

Algorithm 6 walks the fiber chain upward from the accessing context: at the first fiber whose committed view binds key, the access is authorized and that binding is returned; if the walk reaches a fiber that declares key without having committed it, the fiber is not loaded and the access fails; and if it reaches the root without any declaration, the access is rejected as undeclared. This is where the proxy differs from the bare `ctx.get`: `ctx.get(key)` is a lookup against the store that returns the bound value or nothing and never fails, whereas the proxy resolves against the accessing fiber’s own view and enforces the coeffect specification d at the point of use. Reading the view rather than the store is also what Theorem 70 rests on, since it is what keeps a dependency readable to a component whose teardown was triggered by that dependency going away.

This rejection is a *runtime* check performed at the point of access. Because a component’s coeffect specification d is declared statically, the same violation is in principle detectable at compile time, by resolving each `ctx[key]` against the declared d before execution; Section 6.4 discusses how a host language’s type-level dependency declarations and compile-time metaprogramming can carry out exactly this mediation.

5.2. Component Loader

The core library equips *component developers* with imperative primitives for dynamic composition, such as `ctx.effect`, `ctx.use`, and `ctx.set`. A separate concern arises for *application orchestrators*, who assemble pre-existing components into a running system and adjust the composition over its lifetime. The *component loader* addresses this concern by introducing a declarative configuration layer: the orchestrator specifies the desired composition as a persistent data structure, and the loader translates changes to this specification into the corresponding imperative fiber operations.

5.2.1. Declarative Configuration

Section 4 decomposes a running system into fibers, each an instantiation of one component. Everything an instantiation needs can be declared, so an orchestrator can describe a whole system as a *declarative configuration*: a persistent record that the loader realizes as fibers and keeps in step with them.

Entries. A configuration consists of *entries*. Each entry specifies a fiber and manages it, and the binding runs in both directions: the loader responds to a change in an entry's fields by adjusting the fiber, and a component that revises its own configuration or disables itself has the change written back to its entry.

Definition 81. An *entry* declares a single fiber, recording:

- *id* — a stable identifier, used as the reconciliation key when its group's child list changes;
- *url* — the URL of the component module to instantiate;
- *isolate* — an isolation annotation applied to the entry's context;
- *intercept* — an interception annotation applied to the entry's context;
- *config* — the configuration bound into the component to form its effect function *apply*;
- *disabled* — whether the entry is administratively turned off.

An entry can serve as a faithful specification because what supports a fiber is exactly what an entry records. The support set of Definition 74 reads τ , π , d , and p and nothing else, and an entry gives all four: *disabled* gives τ , the entry's parent in the tree gives π , and *url* selects the component which declares d and p . The fields the support set leaves unread are the fiber's runtime state, which an instantiation does not need either, and Lemma 77 identifies the support set with the Active fibers of a quiescent state (Definition 53) as far as each component installs every key it declares (Definition 76).

These entries form a *configuration tree* that is the authoritative record of what the system loads. An entry may be a leaf mapping to a single fiber, or its component may in turn load further components, making the entry a branch node. Cordis provides components for such grouped and nested loading: `@cordisjs/group` takes a list of child entries as its configuration and loads them as a subgroup, and `@cordisjs/include` loads an external configuration file (YAML or JSON) and grafts its entries in as a nested subtree. Both are ordinary components resting on the instantiation primitive of Definition 52 (Algorithm 4), so a nested tree stays within the calculus and the results below hold of it.

Reconciliation. When an entry's record changes, the loader reconciles incrementally rather than tearing the fiber down and rebuilding it wholesale. Reconciling this way is sound for reasons the metatheory supplies.

- Theorem 80 makes the quiescent state a function of the final configuration alone: whatever instantiations and retirements the loader performs on the way, and in whatever order, the system quiesces where a load of the final configuration from scratch would have left it. Which components end up loaded is read off the declarations only as far as each of them installs every key it declares (Definition 76); a component that declares a key and installs it under some configurations alone is one the loader can still reconcile, but the set of loaded components then answers to those configurations as well.
- Theorem 73 proves that the system does quiesce, so a reconciliation is complete once its instantiations and retirements have been issued.
- Corollary 69 puts a departing fiber's contribution to the state at nothing, so rebuilding one entry withdraws what its fiber installed and leaves the fibers around it as they were.

- Theorem 70 lets the entries be instantiated together, with no load order for the orchestrator to arrange: a fiber whose declared keys are not yet provided waits at its L-Begin, and one whose provider leaves is deactivated ahead of it. A dependency therefore constrains when a fiber activates rather than when its module is fetched and evaluated, so the loader loads modules concurrently, where bringing up a large configuration spends its time.

On top of the fiber that an entry declares, the loader dispatches on which of the entry's fields changed and applies the least disruptive operation for each.

- `id, url` — rebuilds the entry, since its identity or its component has changed;
- `isolate` — reassigns the entry's realms (Algorithm 7);
- `intercept` — updated in place, as interception metadata is consulted at read time and needs no reload;
- `config` — handed to the component, which decides how to apply the new payload, typically by diffing it against the previous one and reloading only on a material change. In particular, an `@cordisjs/group` entry's `config` is its list of child entries, so it applies the update as a keyed diff over child `ids`, creating, removing, or updating each child; since updating a surviving child re-enters this same per-field dispatch, group reconciliation and entry update recurse together down the tree;
- `disabled` — unloads the fiber when set and reloads it when cleared.

Managed realms. Isolation in the core derives a child context overriding the realm table ρ at one key (Section 5.1.2), which suffices while the context tree stands still. An entry may be moved between groups at runtime, so the loader manages realms of its own, and the `isolate` field selects between two scoping rules per key. A value of `true` selects a *local* realm, private to the entry and tagged by its `id`, which the entry carries with it wherever it moves; a string selects a *global* realm shared by every entry naming that string, so moving such an entry changes which entries it shares a binding with rather than which realm it belongs to. A realm is discarded once no entry names it.

Reassigning an entry's realms turns on which keys changed realm, whether the entry is itself the provider at a changed key, and which dependents to notify. The middle question is the hard one, since a realm symbol may be shared by several fibers of which only one is the provider. The loader answers it with *delimiters*: one symbol δ_k per key, under which each context stores a tag of its own. A delimiter is written on a context and inherited by its descendants, so the entry's tag and the provider's agree exactly when the two were derived within one isolate scope for k , which is the case in which the binding at k is the entry's own and has to move with it.

Algorithm 7 Isolation realm reassignment

```

1  function patch_isolation(entry,  $\rho'$ )
2       $\rho \leftarrow \text{entry.ctx}[\text{@isolate}]$ 
3       $\text{store} \leftarrow \text{entry.ctx}[\text{@store}]$ 
4       $\Delta \leftarrow \{k \mid \rho(k) \neq \rho'(k)\} \triangleright \text{keys whose realm changes}$ 
5      for  $k$  in  $\Delta$  do
6           $\text{entry.ctx}[\delta_k] \leftarrow \text{fresh tag}$ 
7           $\text{diff}[k] \leftarrow (\rho(k), \rho'(k), \text{entry.ctx}[\delta_k], \text{store}[\rho(k)].\text{fiber.ctx}[\delta_k])$ 
8       $\text{entry.ctx}[\text{@isolate}] \leftarrow \rho'$ 
9       $\text{reload}(\text{entry.fiber})$ 
10     for  $k$  in  $\Delta$  do
```

```

11   (s1, s2, d1, d2) ← diff[k]
12   if d1 = d2 and store[s1] and not store[s2] then ▷ the binding is the entry's own
13       store[s2] ← store[s1]
14       delete store[s1]
15   function affected(fiber, k)
16       (s1, s2, d1, d2) ← diff[k]
17       return fiber.ctx[@@isolate][k] ∈ {s1, s2} and (fiber.ctx[δk] = d1) ≠ (d2 = d1)
18   notify(entry.ctx, Δ, affected) ▷ in place of the realm test of Algorithm 3

```

The test turns on one property of delimiters. The tag under δ_k is written on the entry's context and inherited by every context derived from it, and it is drawn afresh at each reassignment, so for a context γ'

$$\gamma'[\delta_k] = d_1 \iff \gamma' \text{ is derived from the entry's context} \quad (64)$$

Write $\text{own}(\gamma')$ for that condition, of which $d_2 = d_1$ is the instance at the provider. The reassignment moves the contexts satisfying own from s_1 to s_2 and leaves the others where they are, and by the loop above it moves the binding to s_2 exactly when the provider satisfies own . A dependent sees the binding while its own realm at k is the realm the binding sits in. Where own agrees on the dependent and the provider, both move or neither does, so the dependent sees the binding afterwards exactly when it saw it before. Where own separates them, one side moves and the other stays, so the dependent gains or loses the binding. The inequality is that separation, and the membership test drops the dependents resolving k in neither realm, which no part of the move reaches.

5.2.2. Hot Module Replacement

Hot module replacement (HMR) applies the revertible-effect pattern at the *module* level: when source files change, typically during development, the system replaces the affected modules in-place without restarting the process. Because a fiber already bounds all of its component's effects and coeffects, a module that is itself a component can be replaced through fiber operations alone: disposing the old fiber recovers everything the component installed, and a new fiber instantiated from the reloaded module reinstalls it. HMR therefore needs no developer-annotated acceptance boundaries, as opposed to Webpack [48] or Vite [49] HMR.

The `@cordisjs/hmr` component provides the HMR *engine*, which operates in three phases.

Phase 1: Module classification. The engine takes two inputs: the *stashed* set (file URLs whose contents have changed since the last reload) and the *externals* set (modules that cannot be hot-replaced and instead trigger a full restart). Writing `get_imports(url)` for the modules that `url` directly imports, it classifies the changes' dependency subgraph, marking each module accepted or declined:

Algorithm 8 Module classification

```

1  function classify(stashed, externals)
2      accepted ← stashed
3      declined ← externals
4      pending ← ∅
5      for url in stashed do

```

```

6      | pending ← pending ∪ (get_imports(url) \ (accepted ∪ declined))
7      repeat
8          progress ← false
9          for url in pending do
10             if get_imports(url) ∩ accepted ≠ ∅ then
11                 accepted ← accepted ∪ {url}
12                 pending ← pending \ {url}
13                 progress ← true
14             else if get_imports(url) ⊆ declined then
15                 declined ← declined ∪ {url}
16                 pending ← pending \ {url}
17                 progress ← true
18             else
19                 | pending ← pending ∪ (get_imports(url) \ (accepted ∪ declined))
20         until not progress
21     declined ← declined ∪ pending
22     return (accepted, declined)

```

Seeded with the imports of the stashed files, the fixed point accepts a module once one of its imports is accepted and declines one once all of its imports are declined; any module left undecided, caught in an import cycle, defaults to declined.

Phase 2: Stale-entry detection. Using accepted and declined, the engine then filters the component entries down to the *stale* ones, whose dependency tree reaches a changed module. It walks each entry's tree with `get_dependencies`, which collects the transitive imports of a module while respecting declined as a boundary:

Algorithm 9 Stale-entry detection

```

1  function get_dependencies(root, declined)
2      | deps ← ∅
3      function traverse(url)
4          | if url ∈ deps or url ∈ declined then return
5          | deps ← deps ∪ {url}
6          | for child in get_imports(url) do traverse(child)
7      traverse(root)
8      return deps
9  function detect(entries, accepted, declined)
10     | stale_entries ← ∅
11     for entry in entries do
12         | tree ← get_dependencies(entry.url, declined)
13         | if tree ∩ accepted ≠ ∅ then
14             | accepted ← accepted ∪ tree
15             | stale_entries ← stale_entries ∪ {entry}
16     return stale_entries

```

An entry is stale exactly when its tree intersects accepted; that tree is then folded into accepted, so every stale module along it is invalidated in the next phase.

Phase 3: Transactional reload. Finally, the engine reloads the stale entries. It invalidates the accepted modules' caches³, backing up each removed module to enable rollback, then re-imports each stale entry's component module by its url and swaps in a fresh fiber:

Algorithm 10 Transactional module reload

```

1  function reload(ctx, accepted, stale_entries)
2      backup ← invalidate_caches(accepted)
3      try
4          for entry in stale_entries do
5              entry.fiber.dispose()
6              entry.fiber ← ctx.use(import(entry.url), entry.config)
7      catch error
8          restore_caches(backup)
9          for entry in stale_entries do
10             entry.fiber.dispose()
11             entry.fiber ← ctx.use(backup[entry.url], entry.config)
12     throw error

```

The transactional guarantee ensures that the system never enters a half-reloaded state: if any module fails to import (e.g., due to a syntax error), the caches are restored and every stale entry is rebuilt from backup[entry.url], the previous component whose cache was just restored, undoing the swaps already made.

5.3. Case Study: Koishi

Koishi is an open-source chatbot application framework built on Cordis⁴. Over four years of development, it has accumulated over 4000 community-contributed plugins⁵, ranging from instant-messaging (IM) adapters and database drivers to administrative consoles and end-user features. Its scale and diversity make it a representative validation of Cordis's dynamic composability in a production setting.

Expressiveness and generality of the meta-framework. Koishi runs as a server-side bot whose every feature is realized as a plugin over the context primitives of Section 5.1; Koishi itself contributes only the chatbot-domain vocabulary. The same model reappears in a wholly different runtime: Koishi's web console is a second, independent Cordis application whose plugins compose the primitives of the browser and its user interface rather than those of the server. The disparate settings above establish two properties of the model of Section 3. (1) It is expressive: its primitives suffice to carry a complete production system, the host framework supplying only domain vocabulary. (2) It is general: it fixes how effects and coeffects compose while leaving their meaning to each application, and so presupposes neither a particular domain nor a particular runtime.

Temporal composability without cognitive overhead. The plugin systems surveyed in Section 1.2.1 cannot unload an individual extension's effects without restarting the extension

³On Node.js, this means clearing the caches of both the ES module and CommonJS module systems, since a module imported through the ES loader can appear in both.

⁴Koishi currently uses Cordis v3. This paper presents Cordis v4, which refines the effect and coeffect semantics and redesigns the loader; the core compositional model is shared across both versions.

⁵Koishi uses the term *plugin* for the concept this paper formalizes as *component*.

host. Koishi routinely performs this operation: an orchestrator disables a plugin from the console and its effects are reverted in place; during development, the HMR engine re-applies edited plugins on save while preserving cache state and live connections elsewhere in the system. Cordis makes such removal not merely possible but effortless for the plugin author. Because effects performed through the context are tracked and their inverses composed automatically (Section 3.1), even an inexperienced author obtains ordered cleanup for a plugin’s context-mediated effects without writing an uninstall path. This achieves the locality of concern whose absence Section 1.2.1 identifies: correctness that would otherwise rest on each author’s diligence is instead discharged once, by the abstraction.

Spatial composability across an open ecosystem. In contrast to the plugin systems of Section 1.2.1, where inter-plugin dependencies are largely absent, Koishi’s ecosystem exhibits a genuine dependency topology: IM adapters provide access to each messaging platform, database drivers provide persistent storage, and functional plugins declare these as coeffects and access them. Reconfiguring a provider at runtime, such as switching the storage backend or reconnecting an adapter, reactivates only the dependents whose resolved dependency changed (Section 3.2); a plugin whose dependency is unavailable stays inactive until it appears, without erroring. What the case study substantiates is that this composition holds across independently authored code: a plugin and its dependencies are typically written by different authors who coordinate on nothing beyond the coeffect that connects them, so reactive coeffects keep the assembly consistent across an open ecosystem of independent contributors.

Threats to validity. The evidence here is drawn from a single ecosystem in a single host language, so it cannot separate the merits of the paradigm from those of its TypeScript realization or of Koishi’s particular domain, and it is observational rather than a controlled comparison against an alternative architecture. What the case study establishes is thus an existence-and-adoption result rather than a quantitative one; measuring the abstraction’s overhead and its effect on developer productivity against a baseline remains future work.

6. Discussion

The formal model and implementation presented in the preceding sections introduce a programming paradigm for dynamic composability. This section examines how the paradigm extends to broader engineering concerns, and discusses the design tensions and open problems.

6.1. System Boundary

Every effect in Section 3.1 carries an inverse, and what that inverse amounts to is settled by the *system boundary*. The boundary divides the environment a system runs against into two parts. (1) A location lies *inside* when the system is able to modify it exclusively and to restore the state before that modification, so an operation on it is tracked in Γ and can be reverted later. (2) A location lies *outside* when either ability fails, so an operation on it acts as id_Γ and is therefore neither tracked nor reverted. This section develops the properties of this boundary and their consequences for recovery.

Boundaries from coeffects. A coeffect moves the boundary by reifying an external location: it confines every access to that location to a set of operations it provides, each of which it can supply an inverse for, so operations that acted as id_Γ come to be tracked in Γ and reverted. The

boundary is therefore drawn per location rather than per medium, since both aforementioned abilities are properties of a location, and reification changes how a location is accessed while leaving its medium as it was. For example, a memory region lies inside when the system alone writes it, and outside when other processes write it too; a file lies inside when only the system can reach it, as with a scratch file under a private path, and outside when it is a path other programs read or write. Moving the boundary is itself a trade-off, between whether the environment provides revertible semantics for a location and what supplying those semantics costs on every access. We take up the co-design this suggests in Section 6.7.

Acquisition and emission. An operation that reaches outside the boundary generally proceeds in two stages. (1) In the *acquisition* stage, the operation obtains access and installs a record inside the boundary: `open` installs a descriptor that `close` removes, `malloc` reserves a block that `free` releases, `fork` starts a child process that `kill` terminates. The record itself is part of the coeffect that reifies the location, e.g. an entry in a map it keeps, and installing that entry is a revertible effect. That record is at the same time the *channel* along which data can leave. (2) In the *emission* stage, the operation pushes data through that channel, as with the bytes a `write` hands to the file or the datagram a `send` puts on the wire, and the push acts as id_Γ , leaving the data where other parties may read and write it. The two stages therefore fall on opposite sides of the boundary: the acquisition stays inside it, whereas the emission crosses to the outside.

Withholding and compensation. A system that must nonetheless recover from an emission has two approaches available. One is to withhold an emission until the state that produced it is certain to persist, which is the *output commit problem* of rollback-recovery [50]. The other is *compensation* [51]: an action that restores the state up to an equivalence the application supplies, coarser than the $\simeq$ of Definition 33, as in deleting a file that was created or refunding a charge that was made. Such actions compose in the same LIFO order as inverses do, so the composition of Section 3.1 transfers to them. The metatheory does not: the commutation of Definition 65 is proved against $\simeq$ and has to be re-established against the coarser one.

6.2. Service Multiplexing

Dynamic component platforms such as OSGi [52] organize composition around *services*: units of functionality that a provider publishes under an interface and a consumer binds to. The Cordis coeffect model echoes this notion, with a service corresponding to the interface behind a key. Components that provide a service are its *providers*, and components that inject a service are its *consumers*. A single service may be implemented by multiple providers, and this multiplicity can be realized in two forms. (1) *Exclusive binding*: several implementations share one interface but at most one is bound at a time; the orchestrator selects which implementation is bound, and switching between them requires unloading one provider and loading another, momentarily perturbing every consumer's dependency. (2) *Service broker*: a central service that acts as the endpoint for the interface is injected by both the backing providers and the consumers, so that multiple providers coexist and the broker dispatches each request among them. Compared to exclusive binding, the broker absorbs this perturbation: updating a backing provider leaves the broker in place, so consumers see no change to their dependency and no reload is triggered.

The service broker underlies three capabilities: load balancing, rolling updates, and cross-process invocation.

Load balancing. When several providers coexist, the broker distributes requests among them according to a configurable policy (e.g., round-robin, least-loaded, latency-weighted) or

an explicit target named by the consumer. Because providers are ordinary components, they can be added or removed to scale capacity up or down; each provider registers with the broker through a revertible effect, so unloading it reverts the registration and drops it from the broker's routing set automatically.

Rolling updates. Upgrading a service implementation at runtime reduces to a controlled provider transition [53, 54]. To carry out the transition, the new provider is loaded as an additional fiber and registers with the broker; once it becomes `ACTIVE`, traffic is gradually shifted from the old providers to the new one (e.g., by adjusting selection weights), and the old providers are unloaded once they no longer carry in-flight requests. This provider transition turns what is traditionally an infrastructure-level operation (e.g., container orchestration, blue-green deployment) into an application-level composition pattern.

Cross-process invocation. The service broker can also be applied across process boundaries [55]. Each process hosts its own Cordis context with local providers; a coordinating component links them, treating each as a remote provider. Cross-process service access is mediated by an RPC mechanism that preserves the interface, making the distribution transparent to consumers. One caveat is that a cross-process call incurs latency and may fail mid-flight, so exposing it synchronously would block the caller. An interface intended to be exposed across processes must therefore be designed against an asynchronous contract.

6.3. Access Control and Sandboxing

Given an application assembled from independent components, securing the application calls for two complementary mechanisms: (1) constraining what dependencies a component may access, and (2) sandboxing untrusted code from the host environment. Cordis supports the first through dependency declarations and interception; the second requires an external sandbox.

Capability-based access control. The dependency access mechanism (Section 5.1.4) already constitutes a form of access control over proxy-mediated properties: a component can only access dependencies it has declared; an undeclared access raises an error. This is structurally similar to capability-based security [56–58], where authority is conferred by possession of a reference rather than by ambient authority. The inject declaration acts as a capability request, and the context proxy acts as a capability mediator. Since these requests are declared statically, the complete set of proxy-mediated capabilities a component requires is known before it runs, letting the orchestrator review and approve them at load time rather than discovering accesses as they happen.

This mediation generalizes to fine-grained policy through the interception mechanism. Access-control metadata can be carried by contexts or declared by components (Definition 26), and the provider consults it when the dependency is invoked to decide whether a request is permitted. For example, a filesystem dependency may carry metadata declaring which paths a component may read or write, and the provider checks each call against the metadata. Because this interception lives on the context rather than in either party's code, an orchestrator can adjust it to constrain any component's access to a dependency without modifying the provider, e.g., granting read-only database access to a community component whereas a core component retains full access. Moreover, since interception affects only how a dependency is invoked, not whether it is satisfied, it can be installed, reconfigured, or removed at runtime without triggering any reload or perturbing the dependency graph.

Sandboxing untrusted components. When a component’s code cannot be trusted, language-level access control is insufficient, since a malicious component with access to the host runtime can reach the underlying objects directly, rendering such checks moot. Sandboxing requires an execution boundary beyond the reach of language-level means, such as software fault isolation [59], a separate language runtime, a sandboxed process, or a virtualized container [60]. Whatever the mechanism, the untrusted component runs in its own sandboxed context and reaches host-provided dependencies through a bridge, generalizing the cross-process invocation of Section 6.2: the same transparency argument renders this bridged access indistinguishable from local injection. On the host side, the bridge is an ordinary fiber whose capabilities can be attenuated by the access control described above.

6.4. Language Independence and Selection

Although Cordis is implemented in TypeScript, the context paradigm is language-agnostic: spatiotemporal composability is defined only by its two composability dimensions, and thus can be realized in any language that meets certain requirements along both. We analyze these requirements along each dimension in turn.

Temporal composability. At its most basic, temporal composability requires *closures*: a revertible effect pairs an action with an inverse, and that inverse must be captured as a value, along with the state it restores, so it can be replayed on teardown. Beyond this, a component’s code and the side effects of loading it must be introducible and retractable at runtime.

How a language meets this second requirement depends on its execution model. In managed runtimes, this takes the form of a programmatic module registry, where a loaded module can be evicted from the registry and garbage-collected once unreferenced; Node.js, for instance, exposes such a registry.⁶ Native code exposes no module registry, so introduction and retraction take the form of explicit dynamic linking and unlinking (e.g., `dlopen/dlclose` on Unix, `LoadLibrary/FreeLibrary` on Windows) [61], i.e., loading object code into a running process and later detaching it. WebAssembly takes one path or the other depending on its embedder: a module instance is reclaimed by the host’s collector under a managed embedder (e.g., a JavaScript host), or released when a native embedder drops it (e.g., Wasmtime). Across these mechanisms, the revertible effects model treats loading as an effect on the context, with inverses that undo the registration of symbols, types, or handlers the module introduced.

Spatial composability. Spatial composability requires a mechanism for components to declare their dependencies and for the runtime to provide and inject these dependencies. This reduces to a dependency injection (DI) problem [39], which manifests at two levels that differ across languages: how dependencies are *typed* and how their access is *mediated*.

At the type level, the language should provide a way for developers to express well-typed dependency access. A consumer obtains a coeffect by reading its key from the context, so the context type (Section 3.2.1) must record each key’s coeffect. Typeclasses (Haskell) [62] and traits (Rust) [63] achieve this by letting a provider extend the context type from its own module through an instance or `impl` [64]. TypeScript’s module augmentation [65] likewise lets a provider module merge declarations into the context type.

At the runtime level, dependency access must be dynamically mediated: the coeffect behind a key may change as providers are loaded and unloaded, and may be resolved differently across

⁶CommonJS exposes the module cache via `require.cache`; ES modules provide no public eviction API, though modules can still be managed through engine-internal interfaces.